Mothercat Identity API: Frontend Integration Guide

This guide is for the frontend developer integrating the currently available, ongoing Identity and authentication workflows.

Scope

This guide covers these public endpoints:

- **POST** `/api/v1/auth/login` — sign in with staff ID and password.
- **GET** `/api/v1/auth/session` — check the current login session.
- **POST** `/api/v1/auth/logout` — revoke the current session.
- **POST** `/api/v1/auth/password/forgot` — request recovery instructions.
- **POST** `/api/v1/auth/password/reset` — set a new password with a recovery

token.

- **POST** `/api/v1/auth/password/change` — change the signed-in user's

password.

- **POST** `/api/v1/auth/activation/complete` — activate an account and set

its password.

- **POST** `/api/v1/auth/accounts/{staffID}/suspend` — suspend a login account.
- **POST** `/api/v1/auth/accounts/{staffID}/reactivate` — reactivate a

suspended login account.

This guide intentionally excludes the one-time initial SuperAdmin bootstrap and internal service-to-service account provisioning/activation endpoints. Those are not frontend user workflows. Organization endpoints, such as staff record creation, are documented separately in `../contracts/openapi/organization.yaml`.

Environments and base URL

Use the API origin supplied by the backend/deployment owner for each environment. Do not hard-code a

production hostname based on this guide. Configure it as a frontend environment setting, for example:

```
VITE_API_BASE_URL=https://<staging-api-host>
```

The base URL is the API origin only; endpoint paths already begin with `/api/v1`. Never put database URLs, service tokens, bootstrap secrets, or passwords in frontend configuration.

All deployed requests must use HTTPS.

Session and browser requirements

- Successful login sets the `mothercat_session` cookie. It is `Secure`,

`HttpOnly`, `SameSite=Lax`, path `/`, and expires after one hour.

- The frontend must not read, save, copy, or send the session token itself.

The browser manages the HttpOnly cookie.

- For browser `fetch`, set `credentials: "include"` on requests that need the

session cookie, including login, session lookup, logout, password change, and the account suspend/reactivate operations.

- If frontend and API are cross-origin, the API must allow the exact frontend

origin through CORS and allow credentials. Wildcard `Access-Control-Allow-Origin: *` cannot be used with credentialed requests.

- The suspend/reactivate handlers currently require the request `Origin` host

to exactly match the API request host. A frontend hosted on a different origin cannot call those endpoints successfully just by enabling CORS. Serve the frontend/API through the same origin, or coordinate an explicit backend origin-policy change before building those controls.

- `SameSite=Lax` cookies work for same-site deployments (for example,

subdomains under the same registrable domain) when credentials are included. A frontend on a different site may not receive/send this cookie in `fetch` with the current cookie policy. Confirm that frontend and API hosting domains are compatible before integration; a cross-site setup may require a backend cookie-policy change to `SameSite=None; Secure`.

- Because the cookie is always `Secure`, use HTTPS for browser integration.

Plain HTTP development hosts may not store/send it as expected.

- A browser page cannot set the `Cookie` or `Origin` headers manually. The

browser attaches cookies when allowed and generates the `Origin` header for relevant requests.

- Do not treat frontend route guards as authorization. The API remains the

authority and can reject a request at any time.

Common request and error format

Endpoints with JSON request bodies require:

```
Content-Type: application/json
Accept: application/json
```

Send exactly one JSON object. The server rejects unknown JSON fields. A missing/incorrect JSON content type can produce `invalid_request` with a message such as `content-type must be application/json`.

Errors use this envelope:

```
{
  "error": {
    "code": "invalid_request",
    "message": "A human-readable explanation",
    "request_id": "request-correlation-id"
  }
}
```

Use `error.code` to choose UI behavior, show a safe message to the user, and retain `request_id` for support/debugging. Do not display server stack traces or log passwords, tokens, session cookies, or full credential request bodies.

Endpoint details

1. Sign in

`POST /api/v1/auth/login`

Request:

```
{
  "staff_id": "STAFF-100",
  "password": "<user-entered-password>"
```

```
}
```

Success: `200 OK`, JSON response, and a `Set-Cookie` header:

```
{
  "user": {
    "id": "account-id",
    "staff_id": "STAFF-100",
    "display_name": "Example User",
    "email": "user@example.org",
    "role": "Staff",
    "roles": ["Staff"],
    "permissions": ["staff.read"]
  }
}
```

The account ID and role/permission values above are illustrative. Use the returned user object to initialize frontend identity and permission-aware UI. Do not assume the role or permission list is fixed.

Important failures:

- `400 invalid_request`: malformed body or missing `staff_id`/`password`.
- `401 invalid_credentials`: generic invalid staff credentials; do not reveal

which field/account was incorrect.

- `415 invalid_request`: request is not `application/json`.
- `429 rate_limited`: failed-login limit reached.
- `500 internal_error` or `503 service_unavailable`: server/dependency issue.

The limit is five failed attempts per client IP and staff ID per ten-minute process-local window. Show a clear retry-later message for `429`; do not repeatedly retry login automatically.

2. Check the current session

`GET /api/v1/auth/session`

No request body. Include browser credentials. On `200 OK`, response shape is the same `{ "user": ... }` object as login. A missing, expired, revoked, or invalid session returns `401` (`unauthorized` or `invalid_session`). Treat that as signed out and show the sign-in flow; it is not normally a server failure.

Call this when the application starts or reloads to restore the UI session. Do not trust a cached frontend user object instead of checking the server.

3. Sign out

`POST /api/v1/auth/logout`

No request body. Include browser credentials. `200 OK` returns:

```
{ "status": "logged_out" }
```

The server revokes the current session when present and clears the session cookie. Clear frontend user state after success. On `503`, the server could not confirm revocation; show an error and allow the user to retry rather than claiming a successful logout.

4. Request password recovery

`POST /api/v1/auth/password/forgot`

Request:

```
{ "staff_id": "STAFF-100" }
```

Success: `202 Accepted`:

```
{
  "status": "accepted",
  "message": "If the account is eligible, recovery instructions will be sent."
}
```

Always show the generic accepted confirmation. Do not tell the user whether the staff ID exists. Limit: ten recovery/activation workflow requests per client IP per ten-minute process-local window.

Possible errors: `400 invalid_request`, `429 rate_limited`, and `503 email_delivery_disabled` or `service_unavailable`.

5. Complete password recovery

`POST /api/v1/auth/password/reset`

Request:

```
{
  "token": "<token-from-recovery-instructions>",
  "password": "<new-password>",
  "confirm_password": "<same-new-password>"
}
```

The token is single-use. The password must be 14-72 bytes and contain at least three of: uppercase letters, lowercase letters, digits, and other characters. The server also requires the two password fields to match.

Success: `200 OK`:

```
{ "status": "password_reset" }
```

Possible errors: `400 invalid_request`, `invalid_token` (invalid, expired, or already used), or `weak_password`; `429 rate_limited`; `500 internal_error`; and `503 service_unavailable`.

6. Change password while signed in

`POST /api/v1/auth/password/change`

Requires the current session cookie. Include browser credentials.

Request:

```
{
  "current_password": "<current-password>",
  "password": "<new-password>",
  "confirm_password": "<same-new-password>"
}
```

The new password must follow the 14-72 byte and three-character-class rules described above. The current password must be valid and the new password fields must match.

Success: `200 OK`:

```
{
  "status": "password_changed",
  "message": "Sign in again with your new password."
}
```

The current session is revoked and its cookie cleared. Clear local authenticated state and send the user to sign in again.

Possible errors: `400 invalid_request` or `weak_password`; `401 invalid_credentials`; `500 internal_error`; and `503 service_unavailable`.

7. Complete account activation

`POST /api/v1/auth/activation/complete`

Use the single-use token supplied through the account activation workflow:

```
{
  "token": "<token-from-activation-instructions>",
  "password": "<new-password>",
  "confirm_password": "<same-new-password>"
}
```

The new password must follow the same password rules as password reset.

Success: `200 OK`:

```
{ "status": "activated" }
```

Possible errors: `400 invalid_request`, `invalid_token`, or `weak_password`; `429 rate_limited`; `500 internal_error`; and `503 service_unavailable`.

8. Suspend an account

`POST /api/v1/auth/accounts/{staffID}/suspend`

Requires all of the following:

- A valid session cookie.
- The caller's `permissions` include `settings.manage`.
- A browser-generated `Origin` matching the API's scheme and host.
- `credentials: "include"` in browser fetch.

No request body. Replace `{staffID}` with the target user's official staff ID. On success, `200 OK`:

```
{ "status": "suspended", "staff_id": "STAFF-100" }
```

Suspension revokes the target account's current sessions and invalidates outstanding activation/recovery tokens. It does not change employment status. The last active SuperAdmin cannot be suspended.

Possible errors: `400 invalid_staff_id`; `401 unauthorized` or `invalid_session`; `403 invalid_origin` or `permission_denied`; `404 account_not_found`; `409 last_active_super_admin` or `account_state_conflict`; and `503 account_operation_unavailable` / `identity_unavailable`.

9. Reactivate an account

`POST /api/v1/auth/accounts/{staffID}/reactivate`

Uses the same session, `settings.manage` permission, matching `Origin`, and browser credential requirements as suspend. No request body.

Success: `200 OK`:

```
{ "status": "reactivated", "staff_id": "STAFF-100" }
```

Reactivation clears the suspension; the user must sign in again. Possible errors: `400 invalid_staff_id`; `401 unauthorized` or `invalid_session`; `403 invalid_origin` or `permission_denied`; `404 account_not_found`; `409 account_state_conflict`; and `503 account_operation_unavailable` / `identity_unavailable`.

Browser `fetch` example

Use a shared helper so JSON headers, credentials, and error parsing are consistent. Do not stringify `undefined` request bodies; call bodyless endpoints without a `body`.

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

type IdentityUser = {
  id: string;
  staff_id: string;
  display_name: string;
  email: string;
  role: string;
  roles: string[];
  permissions: string[];
};

type ApiError = {
  error?: {
    code?: string;
    message?: string;
    request_id?: string;
  };
};

async function apiRequest<T>(
  path: string,
  options: {
    method: "GET" | "POST";
    body?: unknown;
  },
): Promise<T> {
  const headers: Record<string, string> = {
    Accept: "application/json",
  };

  if (options.body !== undefined) {
```

```
    headers["Content-Type"] = "application/json";
  }

  const response = await fetch(`${API_BASE_URL}${path}`, {
    method: options.method,
    headers,
    credentials: "include",
    ...(options.body === undefined
      ? {}
      : { body: JSON.stringify(options.body) }),
  });

  if (!response.ok) {
    const error = (await response.json().catch(() => null)) as ApiError | null;
    throw {
      status: response.status,
      code: error?.error?.code ?? "request_failed",
      message: error?.error?.message ?? "The request could not be completed.",
      requestId: error?.error?.request_id,
    };
  }

  return (await response.json()) as T;
}
```

Example calls:

```
const loginResult = await apiRequest<{ user: IdentityUser }>(
  "/api/v1/auth/login",
  {
    method: "POST",
    body: { staff_id: staffId, password },
  },
);

const sessionResult = await apiRequest<{ user: IdentityUser }>(
  "/api/v1/auth/session",
  { method: "GET" },
);

await apiRequest<{ status: string }>("/api/v1/auth/logout", {
  method: "POST",
});
```

The browser automatically supplies `Origin` for the account-state `POST`s; frontend JavaScript should not try to set that forbidden header.

Suggested frontend flow

1. On app startup, request `GET /api/v1/auth/session`.
2. On `200`, store the returned user in application state. On `401`, display

the signed-out state. On network/`5xx` errors, show a retry/error state rather than treating the response as a successful login.

1. Submit the login form to `/login`; on success store the returned user and

navigate to the authenticated area. The cookie is set by the browser.

1. On logout, call `/logout`, then clear local user state after server success.
2. For protected administrator controls, use returned permissions to decide

whether to show the UI, but still handle API `401`/`403` responses.

1. For recovery and activation, keep tokens out of logs/analytics and avoid

persisting them longer than needed to submit the completion request.

Rate limits and client behavior

- Login: five failed attempts per client IP and staff ID per ten minutes.
- Forgot password, reset, and activation completion: ten requests per client IP

per ten minutes.

- Limits are in-memory, process-local, reset on process restart, and not shared

between replicas.

- A `429` response has `error.code: "rate_limited"`. Tell the user to wait and

avoid automatic rapid retries. The API does not currently document a `Retry-After` header, so do not depend on one.

Test checklist for frontend integration

- Login sends JSON and receives a user plus session cookie.
- Reloading the app calls session lookup and restores a valid session.
- Invalid credentials show a generic sign-in error.
- Logout clears the application state and invalidates the session.

- Forgot-password always shows the generic accepted message.
- Reset and activation handle invalid/expired/used token and weak-password

responses.

- Password change clears the session and requires another login.
- Users without `settings.manage` cannot use account suspend/reactivate.
- Suspend/reactivate include credentials and succeed only from an allowed

matching origin.

- `429`, network failures, and server errors are surfaced without reporting

false success.

- Verify cookies in the browser against the actual staging frontend/API domain

arrangement. If the cookie is blocked because requests are cross-site, stop and coordinate a deployment/cookie configuration change with the backend owner.

Contract source

The detailed machine-readable contract is `../contracts/openapi/identity.yaml`. If this guide and the deployed API differ, report the discrepancy to the backend owner and update the contract; do not infer undocumented behavior.